

STRUCT Course

STRING courseNumber

STRING courseName

VECTOR<STRING> prerequisites

END STRUCT

CLASS CourseBST

STRUCT Node

Node* left

Node* right

Course data

END STRUCT

Node* root = null

FUNCTION Insert(course)

root = InsertRecursive(root, course)

END FUNCTION

FUNCTION InsertRecursive(node, course)

```
IF node == null
    CREATE Node newNode
    newNode.data = course
    newNode.left = null
    newNode.right = null
    RETURN newNode
END IF
```

```
IF course.courseNumber < node.data.courseNumber
    node.left = InsertRecursive(node.left, course)
ELSE
    node.right = InsertRecursive(node.right, course)
END IF
```

```
RETURN node
```

```
END FUNCTION
```

```
FUNCTION Search(courseNumber)
    RETURN SearchRecursive(root, courseNumber)
END FUNCTION
```

```
FUNCTION SearchRecursive(node, courseNumber)
```

```
IF node == null
```

```
    RETURN null
```

```
END IF
```

```
IF node.data.courseNumber == courseNumber
```

```
    RETURN node
```

```
END IF
```

```
IF courseNumber < node.data.courseNumber
```

```
    RETURN SearchRecursive(node.left, courseNumber)
```

```
ELSE
```

```
    RETURN SearchRecursive(node.right, courseNumber)
```

```
END IF
```

```
END FUNCTION
```

```
FUNCTION PrintAllCourses()
```

```
    IF root == null
```

```
        PRINT "No courses available"
```

```
        RETURN
```

```
    END IF
```

```
    InOrder(root)
```

```
END FUNCTION
```

```
FUNCTION InOrder(node)
```

```
    IF node != null
```

```
        InOrder(node.left)
```

```
        PRINT node.data.courseNumber + ", " + node.data.courseName
```

```
        InOrder(node.right)
```

```
    END IF
```

```
END FUNCTION
```

```
FUNCTION PrintCourse(courseNumber)
```

```
    DECLARE node = Search(courseNumber)
```

```
    IF node == null
```

```
        PRINT "Course not found"
```

```
        RETURN
```

```
    END IF
```

```
    PRINT node.data.courseNumber + ", " + node.data.courseName
```

IF node.data.prerequisites is empty

 PRINT "No prerequisites"

ELSE

 PRINT "Prerequisites:"

 FOR EACH prereq IN node.data.prerequisites

 PRINT prereq

 END FOR

END IF

END FUNCTION

END CLASS

FUNCTION LoadFileIntoBST(fileName, bst)

 OPEN file using fileName

 IF file fails to open

 PRINT "FATAL ERROR: could not open file"

 RETURN

 END IF

```
DECLARE VECTOR<Course> tempCourses
```

```
DECLARE SET validCourseNumbers
```

```
// =====
```

```
// PASS 1: READ FILE ONLY
```

```
// =====
```

```
FOR EACH line IN file
```

```
    IF line is empty
```

```
        PRINT "FATAL ERROR: empty line detected"
```

```
        CLOSE file
```

```
        RETURN
```

```
    END IF
```

```
    SPLIT line by comma INTO tokens
```

```
    IF tokens.size < 2
```

```
        PRINT "FATAL ERROR: invalid format on line"
```

```
        CLOSE file
```

```
        RETURN
```

```
    END IF
```

```
    CREATE Course course
```

```
    course.courseNumber = TRIM(tokens[0])
```

```
course.courseName = TRIM(tokens[1])
```

```
FOR i FROM 2 TO tokens.size - 1
```

```
    ADD TRIM(tokens[i]) TO course.prerequisites
```

```
END FOR
```

```
ADD course TO tempCourses
```

```
ADD course.courseNumber TO validCourseNumbers
```

```
END FOR
```

```
CLOSE file
```

```
// =====
```

```
// PASS 2: VALIDATE
```

```
// =====
```

```
FOR EACH course IN tempCourses
```

```
    FOR EACH prereq IN course.prerequisites
```

```
        IF prereq NOT IN validCourseNumbers
```

```
            PRINT "FATAL ERROR: Missing prerequisite " + prereq +
```

```
                " for course " + course.courseNumber
```

```
            RETURN
```

```
        END IF
```

END FOR

END FOR

// =====

// PASS 3: INSERT INTO BST

// =====

FOR EACH course IN tempCourses

 bst.Insert(course)

END FOR

END FUNCTION

START MAIN

DECLARE CourseBST bst

DECLARE INTEGER userChoice

DECLARE STRING courseNumber

WHILE true

 PRINT "1. Load file"

 PRINT "2. Print all courses"

 PRINT "3. Search course"

PRINT "9. Exit"

GET userChoice

SWITCH userChoice

CASE 1:

LoadFileIntoBST("Example.csv", bst)

PRINT "Courses loaded into BST"

BREAK

CASE 2:

bst.PrintAllCourses()

BREAK

CASE 3:

PRINT "Enter course number:"

GET courseNumber

bst.PrintCourse(courseNumber)

BREAK

CASE 9:

PRINT "Have a nice day!"

RETURN

DEFAULT:

PRINT "Invalid choice"

BREAK

END SWITCH

END WHILE

END MAIN